上海科技大学
ShanghaiTech University

CS240 Algorithm Design and Analysis

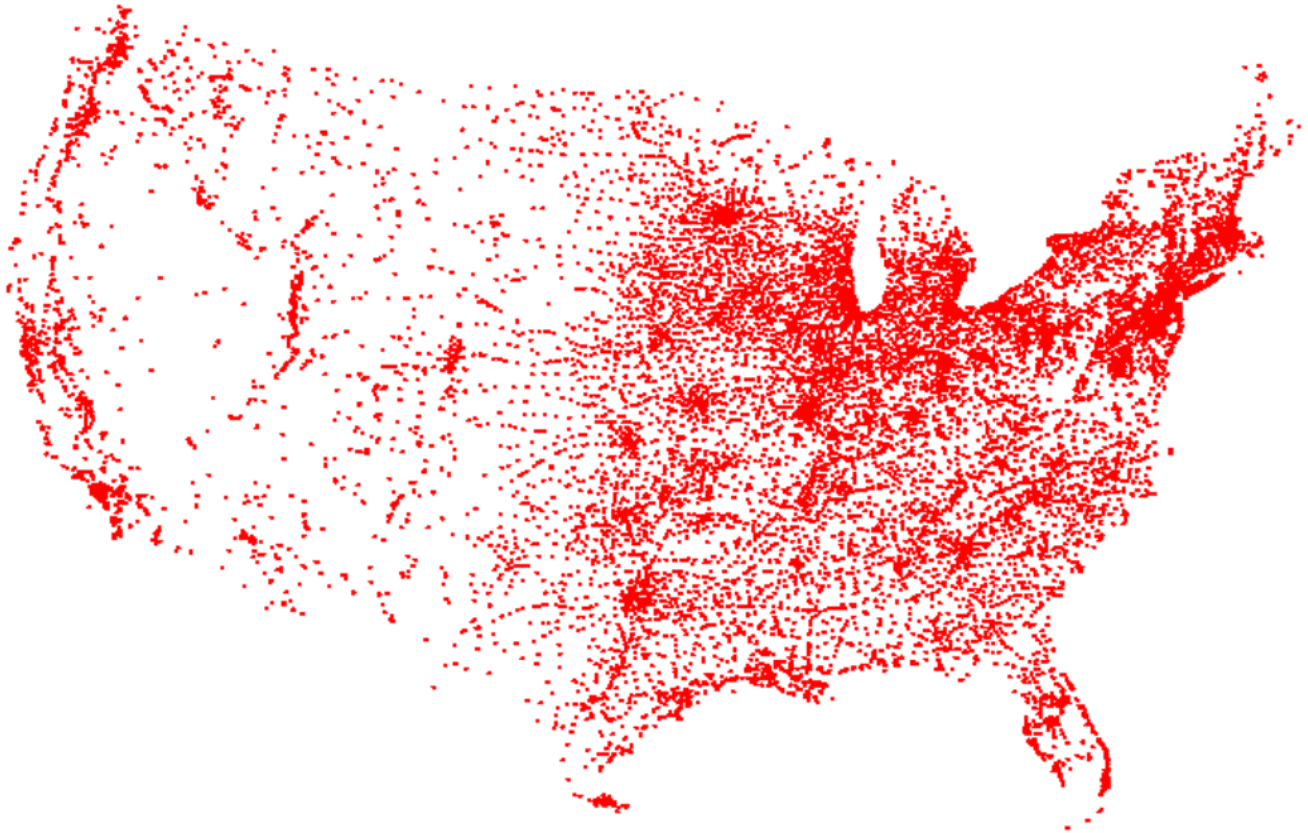
Lecture 13

NP and Computational Intractability (Cont.)

Quan Li
Fall 2025
2025.10.30

Traveling Salesperson Problem

- **TSP.** Given a set of n cities and a pairwise distance function d(u, v), is there a tour of length ≤ D?



All 13,509 cities in US with a population of at least 500
Reference: http://www.tsp.gatech.edu

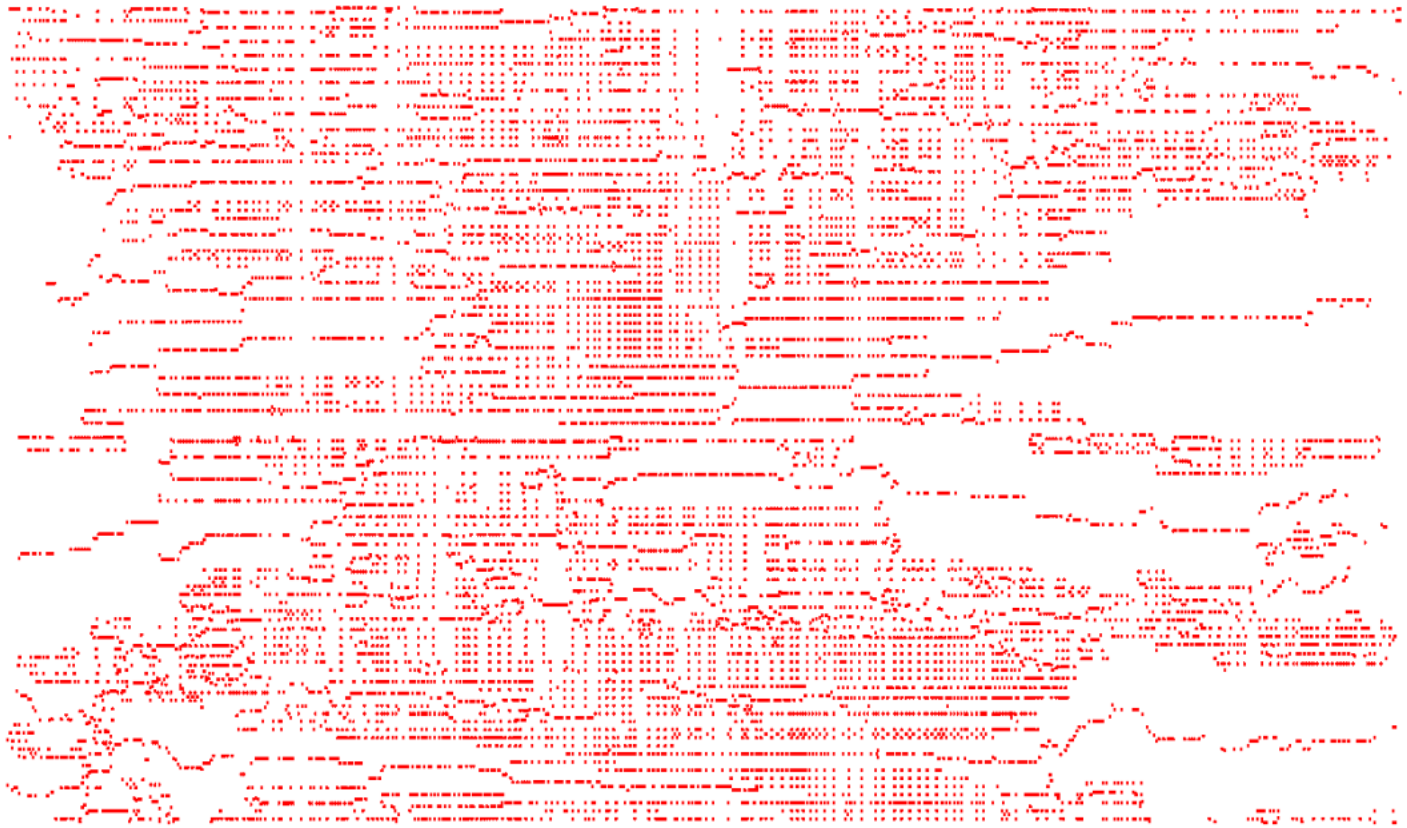
Traveling Salesperson Problem

- **TSP.** Given a set of n cities and a pairwise distance function d(u, v), is there a tour of length ≤ D?

Optimal TSP tour
Reference: http://www.tsp.gatech.edu

Traveling Salesperson Problem

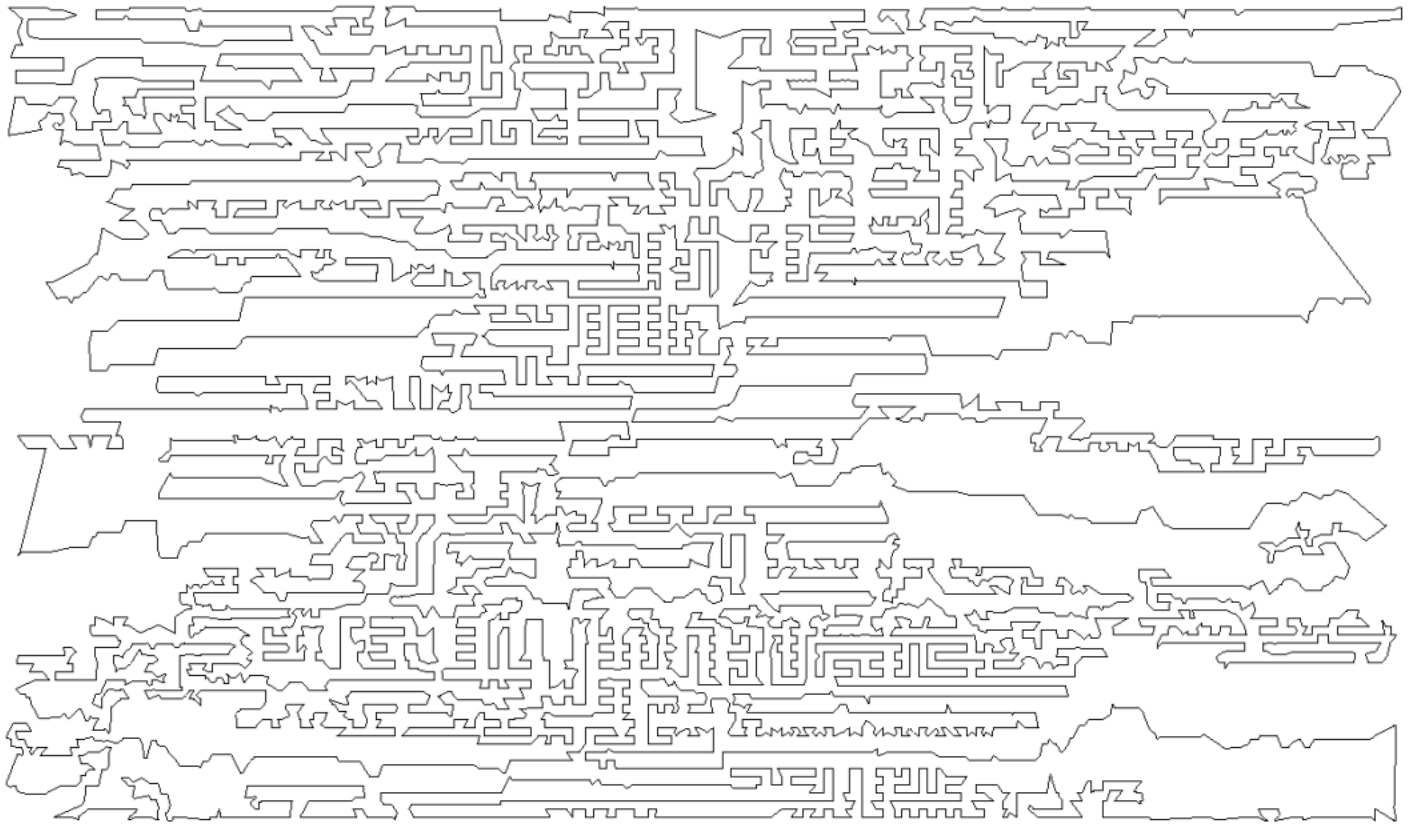
- **TSP.** Given a set of n cities and a pairwise distance function d(u, v), is there a tour of length ≤ D?



11,849 holes to drill in a programmed logic array
Reference: http://www.tsp.gatech.edu

Traveling Salesperson Problem

- **TSP.** Given a set of n cities and a pairwise distance function d(u, v), is there a tour of length ≤ D?



Optimal TSP tour
Reference: http://www.tsp.gatech.edu

Traveling Salesperson Problem

- **TSP.** Given a set of n cities and a pairwise distance function d(u, v), is there a tour of length ≤ D?
- **HAM-CYCLE:** given a graph G = (V, E), does there exists a simple cycle that contains every node in V?
- **Claim.** HAM-CYCLE $\leq_p$ TSP
- **Pf.**
  - Given instance G = (V, E) of HAM-CYCLE, create n cities with distance function

$$d(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ 2 & \text{if } (u, v) \notin E \end{cases}$$

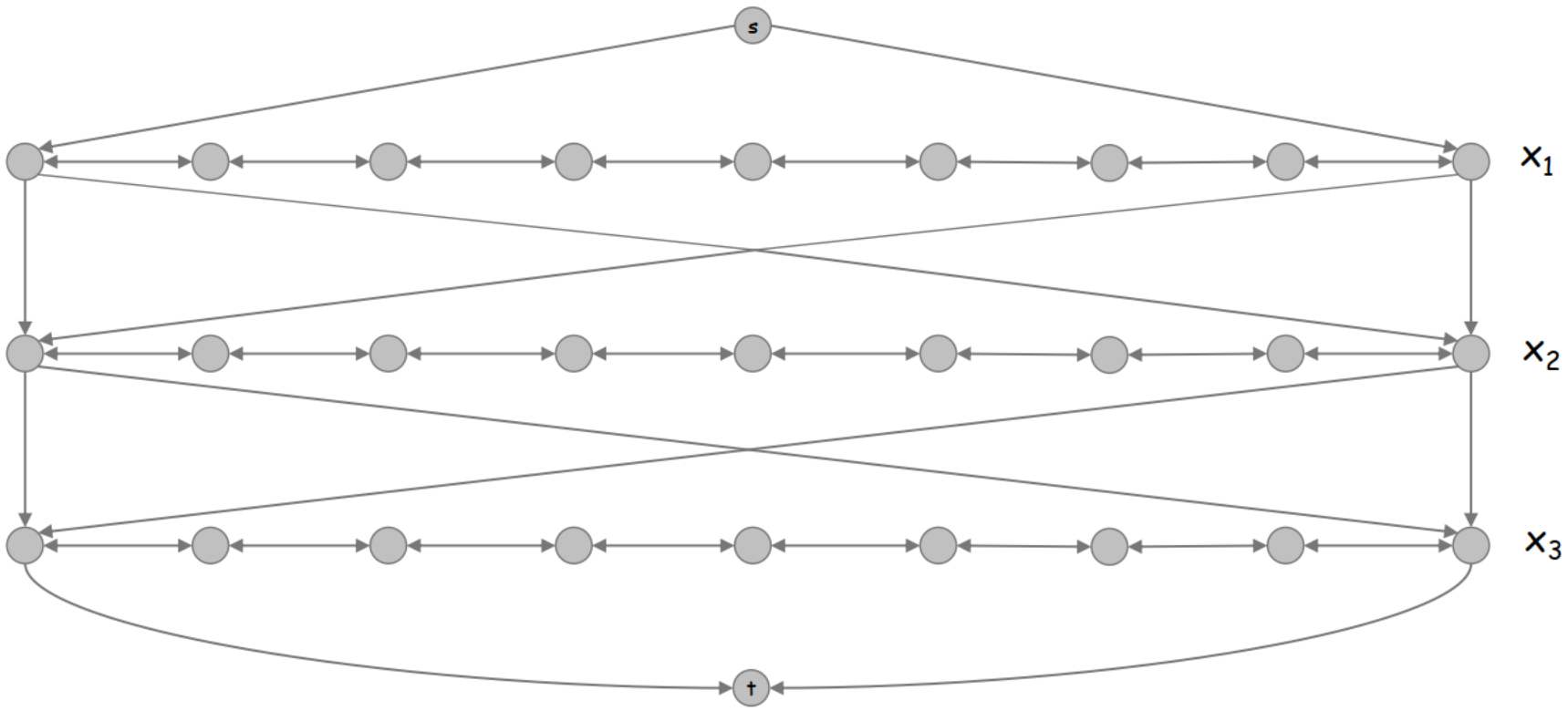
TSP instance has tour of le

Longest Path

- **SHORTEST-PATH.** Given a digraph G = (V, E), does there exist a simple path of length **at most** k edges?
- **LONGEST-PATH.** Given a digraph G = (V, E), does there exist a simple path of length **at least** k edges?

Longest Path

- **Claim.** 3-SAT $\leq_p$ LONGEST-PATH
- **Pf.** Redo proof for DIR-HAM-CYCLE, ignoring back-edge from t to s



Sequencing Problems

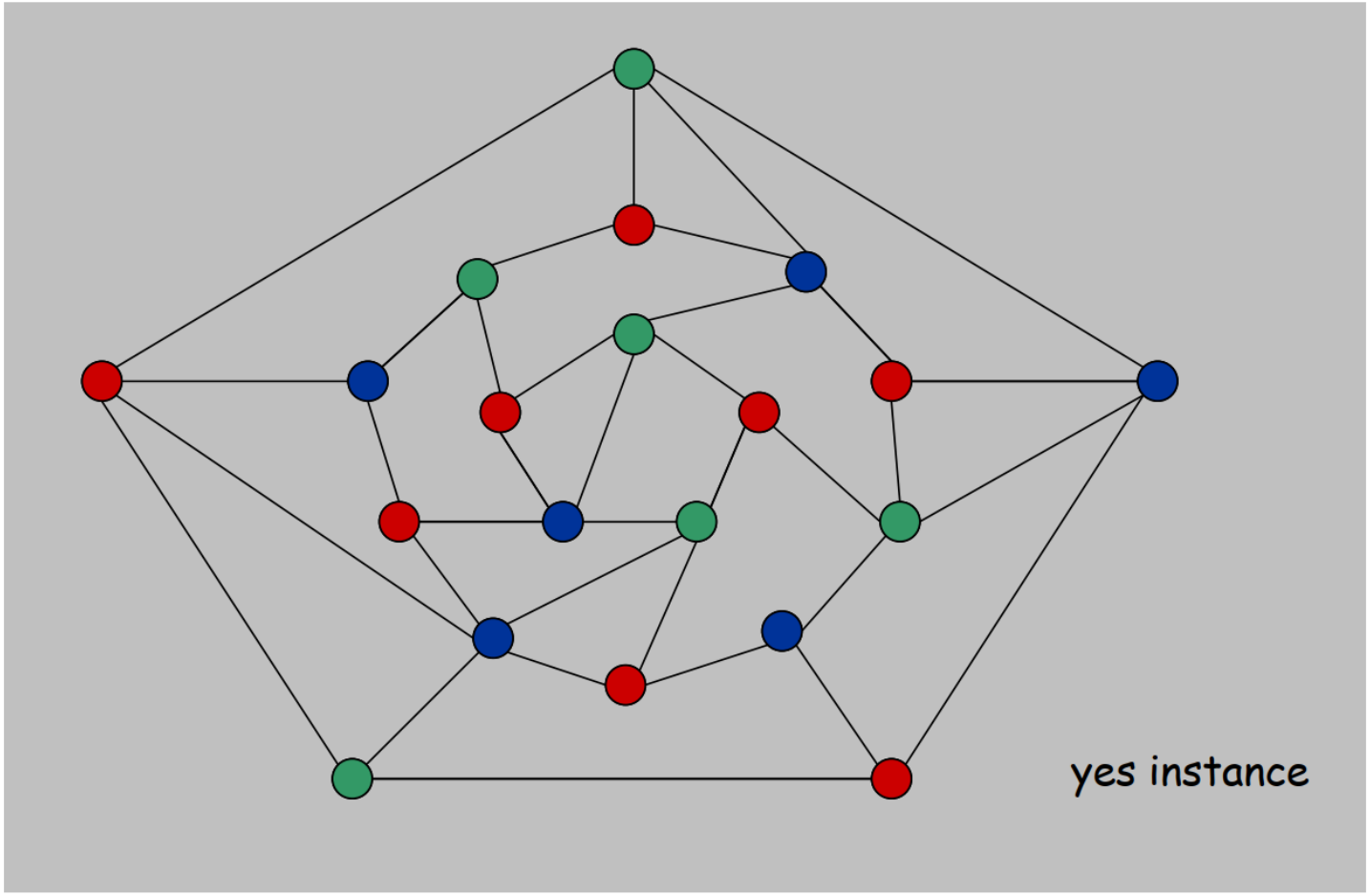
- **Basic genres**
  - Packing problems: SET-PACKING, INDEPENDENT SET
  - Covering problems: SET-COVER, VERTEX-COVER
  - Constraint satisfaction problems: SAT, 3-SAT
  - Sequencing problems: HAMILTONIAN-CYCLE, TSP
  - **Partitioning problems: 3-COLOR, 3D-MATCHING**
  - Numerical problems: SUBSET-SUM, KNAPSACK

Graph Coloring

- **Basic genres**
  - Packing problems: SET-PACKING, INDEPENDENT SET
  - Covering problems: SET-COVER, VERTEX-COVER
  - Constraint satisfaction problems: SAT, 3-SAT
  - Sequencing problems: HAMILTONIAN-CYCLE, TSP
  - **Partitioning problems: 3-COLOR, 3D-MATCHING**
  - Numerical problems: SUBSET-SUM, KNAPSACK

3-Colorability

- **3-COLOR:** Given an undirected graph G, does there exist a way to color the nodes red, green, and blue so that no adjacent nodes have the same color?



Application: Register Allocation

- **Register allocation.** Assign program variables to machine register so that no more than k registers are used and no two program variables that are needed at the same time are assigned to the same register

- **Interference graph.** Nodes are program variables names, edge between u and v if there exists an operation where both u and v are “live” at the same time

- **Observation.** [Chaitin 1982] Can solve register allocation problem iff interference graph is k-colorable

- **Fact.** 3-COLOR $\leq_p$ k-REGISTER-ALLOCATION for any constants $k \geq 3$

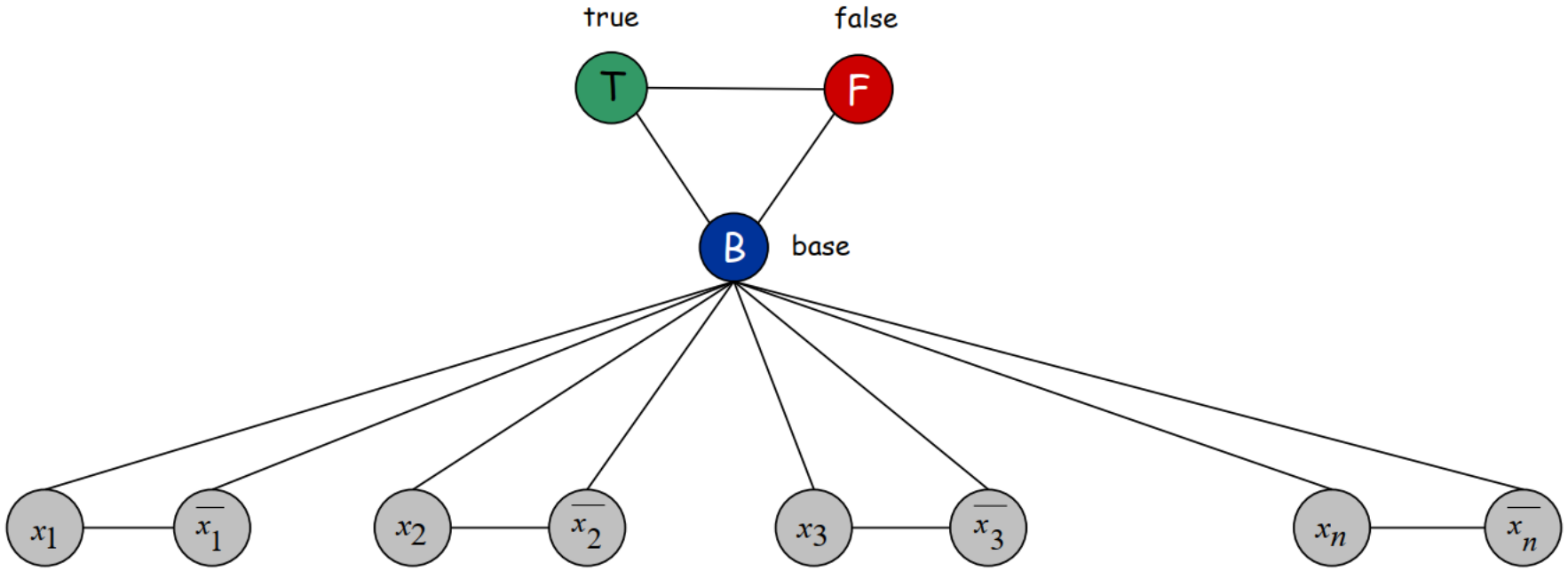
3-Colorability

- **Claim.** 3-SAT $\leq_p$ 3-COLOR
- **Pf.** Given 3-SAT instance ϕ, we construct an instance of 3-COLOR that is 3-colorable iff ϕ is satisfiable

- **Recipe to establish NP-completeness of problem Y**
  - Step 1. Show that Y is in NP
  - Step 2. Choose an NP-complete problem X
  - Step 3. Prove that X $\leq_p$ Y

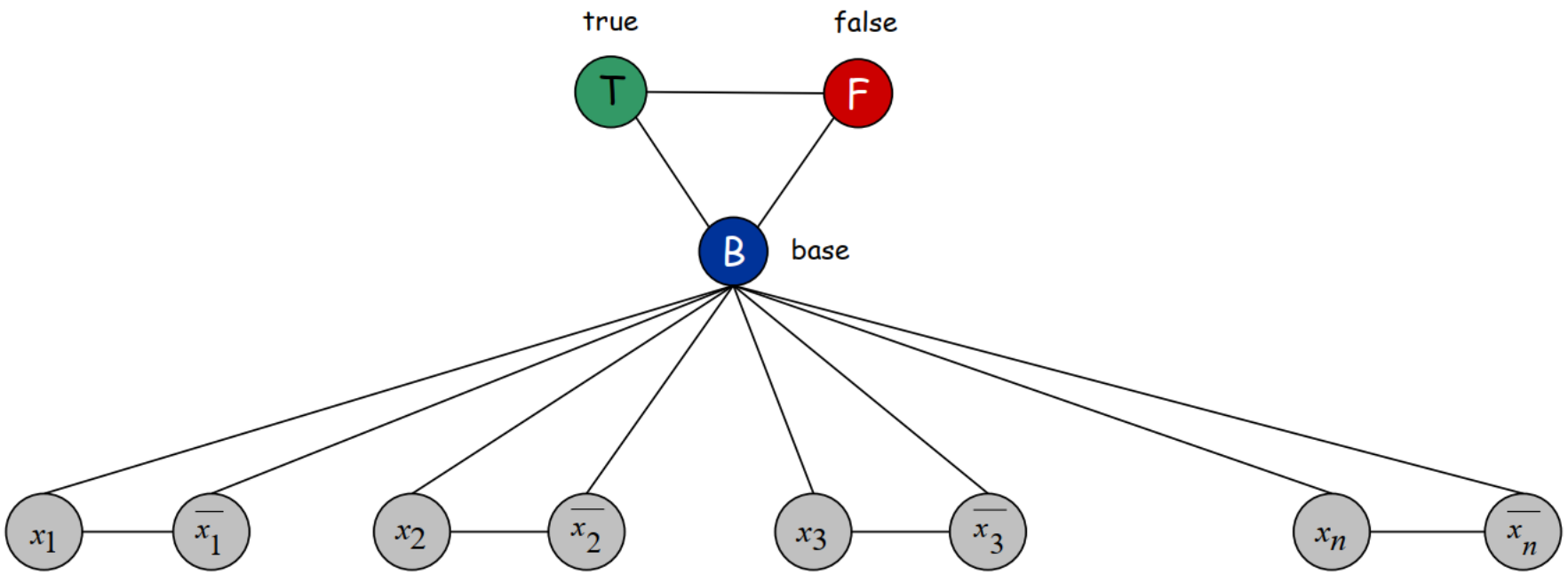
3-Colorability

- **Construction**
  I. For each literal, create a node
  II. Connect each literal to its negation
  III. Create 3 new nodes T, F, B; connect them in a triangle
  IV. Connect each literal to B
  V. For each clause, add gadget of 6 nodes and 13 edges (to be described later)



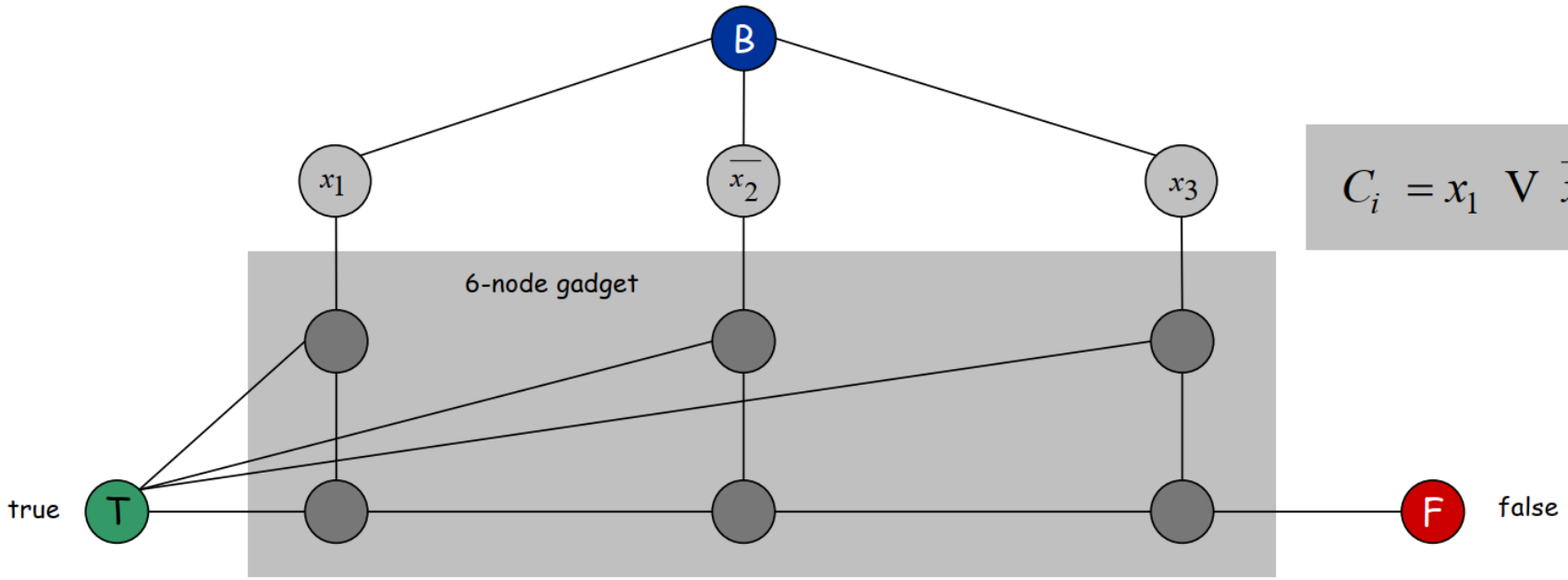
3-Colorability

- **Claim**. Graph is 3-colorable iff ϕ is satisfiable
- **Pf.** → Suppose graph is 3-colorable
  - Set all the literals with T color to true
  - Ensures each literal is T or F
  - Ensures a literal and its negation are opposites



3-Colorability

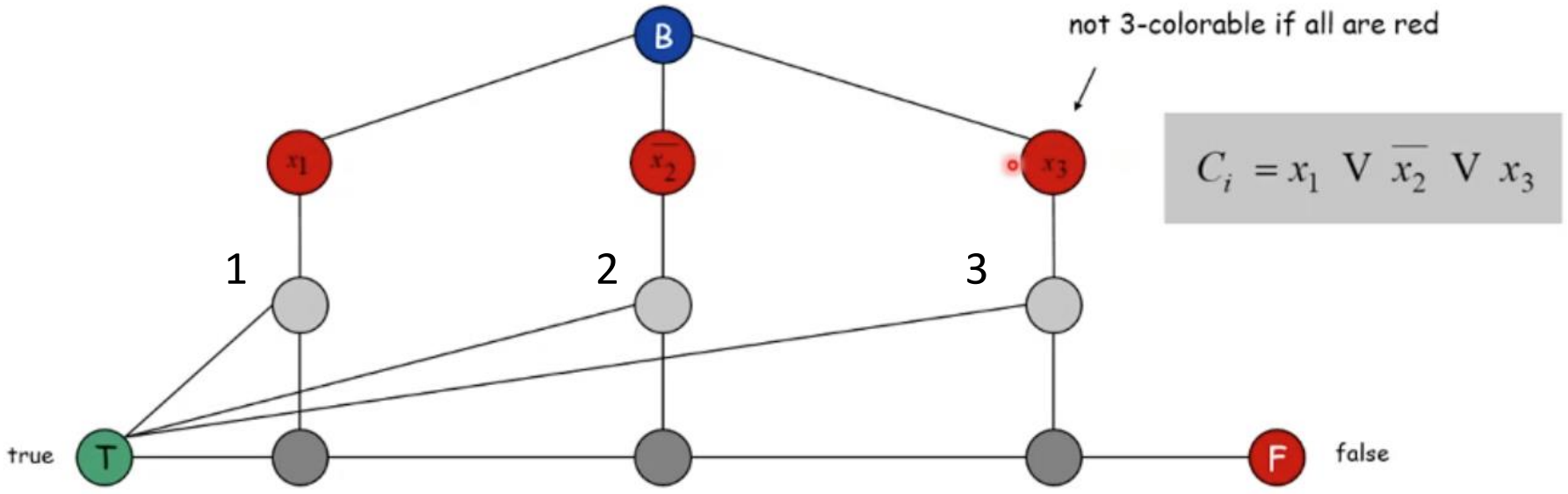
- **Claim**. Graph is 3-colorable iff ϕ is satisfiable
- **Pf.** → Suppose graph is 3-colorable
  - Set all the literals with T color to true
  - Ensures each literal is T or F
  - Ensures a literal and its negation are opposites
  - Ensures at least one literal in each clause is T



$$C_i = x_1 \vee \overline{x_2} \vee x_3$$

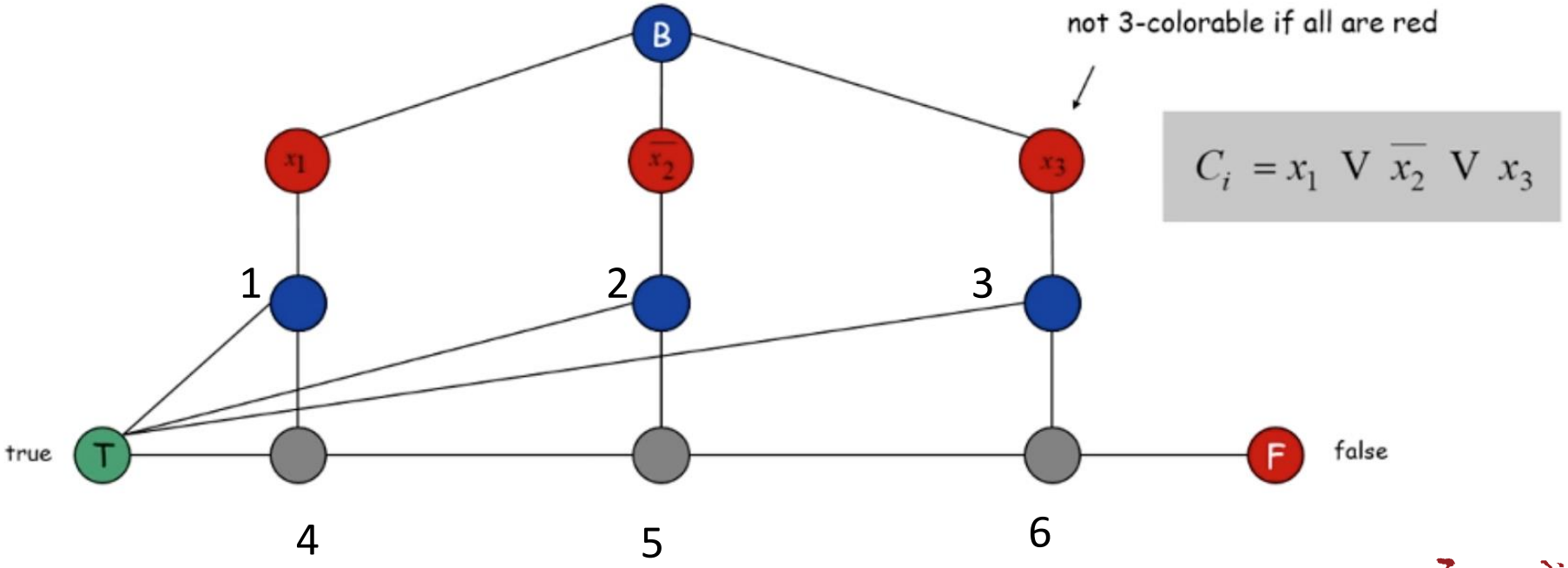
3-Colorability

- **Claim.** Graph is 3-colorable iff ϕ is satisfiable
- **Pf.** → Suppose graph is 3-colorable
  - Set all the literals with T color to true
  - Ensures each literal is T or F
  - Ensures a literal and its negation are opposites
  - Ensures at least one literal in each clause is T



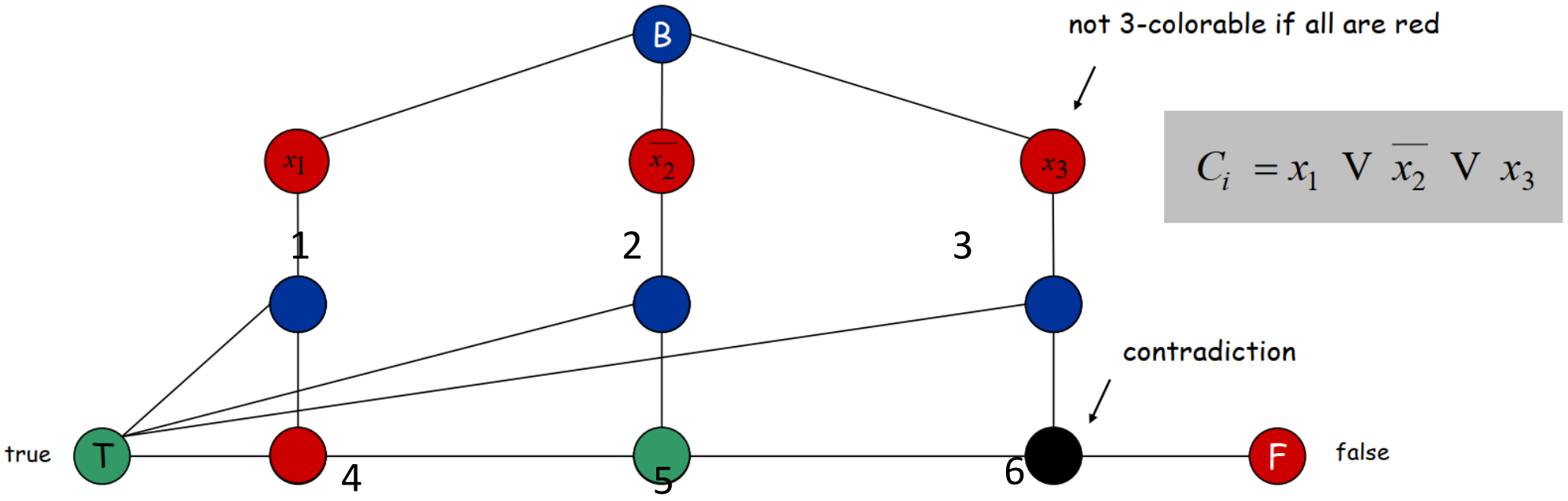
3-Colorability

- **Claim.** Graph is 3-colorable iff ϕ is satisfiable
- **Pf.** → Suppose graph is 3-colorable
  - Set all the literals with T color to true
  - Ensures each literal is T or F
  - Ensures a literal and its negation are opposites
  - Ensures at least one literal in each clause is T



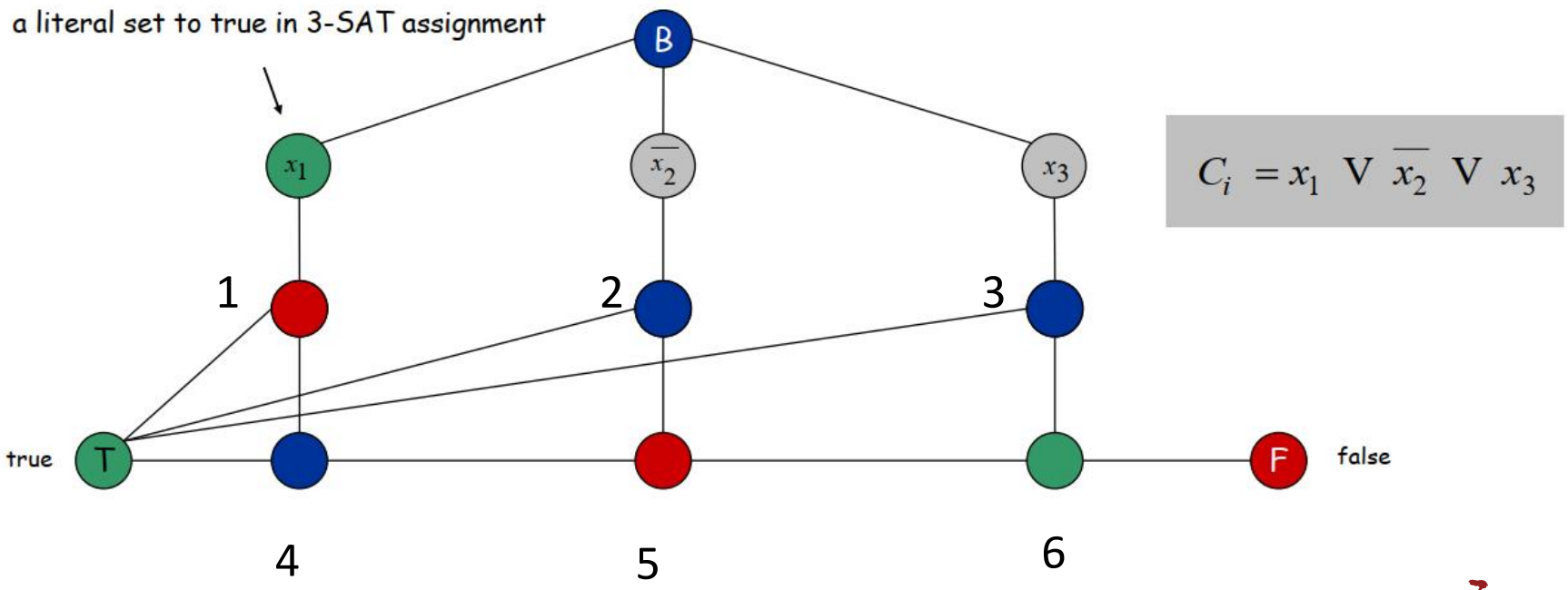
3-Colorability

- **Claim.** Graph is 3-colorable iff ϕ is satisfiable
- **Pf.** → Suppose graph is 3-colorable
  - Set all the literals with T color to true
  - Ensures each literal is T or F
  - Ensures a literal and its negation are opposites
  - Ensures at least one literal in each clause is T



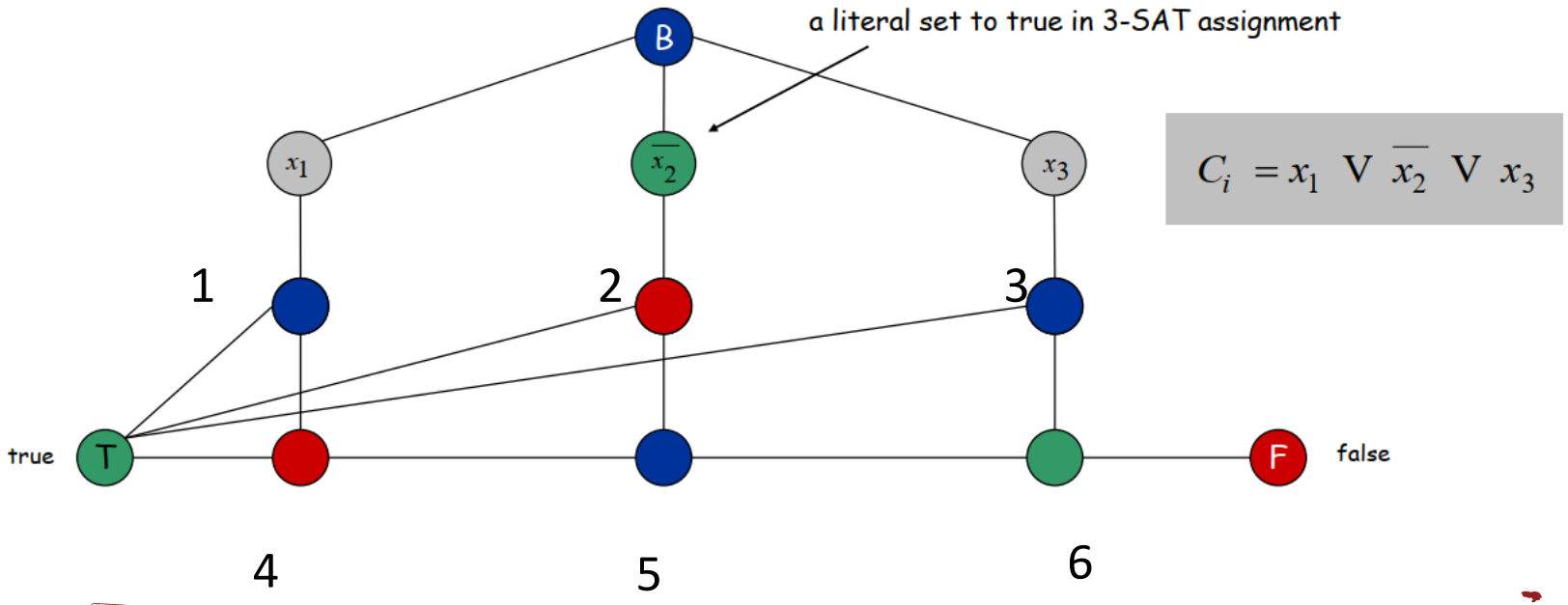
3-Colorability

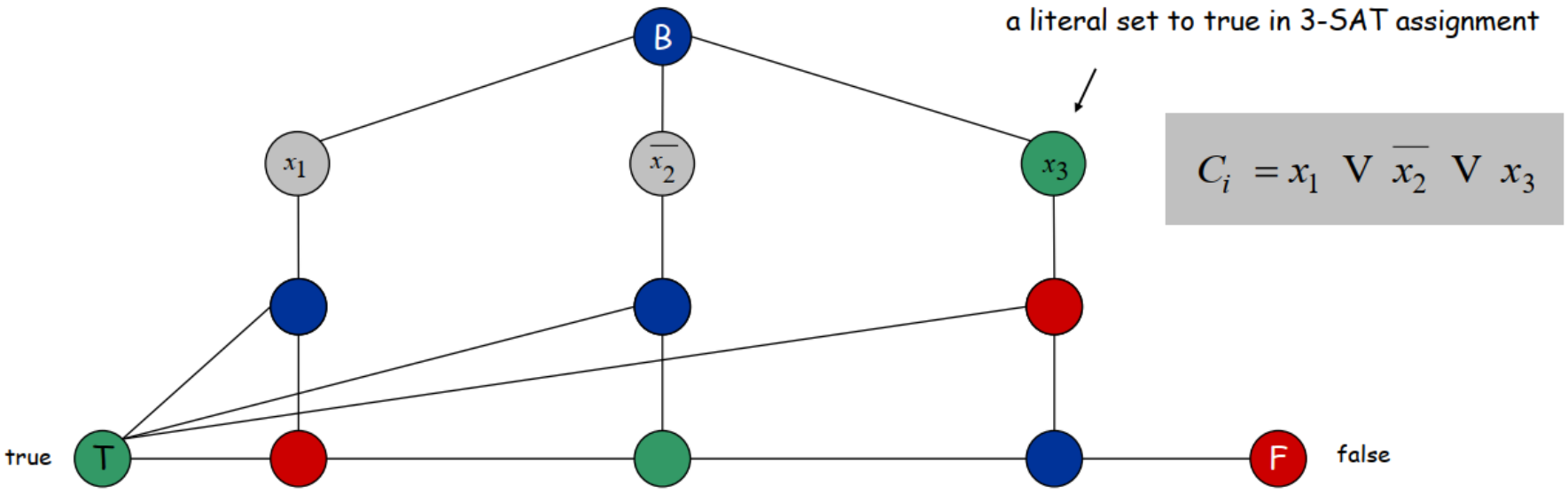
- **Claim.** Graph is 3-colorable iff ϕ is satisfiable
- **Pf.** ← Suppose 3-SAT formula ϕ is satisfiable
  - Color all true literals T
  - Color node below green node F, and node below that B
  - Color remaining middle row nodes B
  - Color remaining bottom nodes T or F as forced



3-Colorability

- **Claim.** Graph is 3-colorable iff ϕ is satisfiable
- **Pf.** ← Suppose 3-SAT formula ϕ is satisfiable
  - Color all true literals T
  - Color node below green node F, and node below that B
  - Color remaining middle row nodes B
  - Color remaining bottom nodes T or F as forced



3-Colorability

- **Claim.** Graph is 3-colorable iff ϕ is satisfiable
- **Pf.** ← Suppose 3-SAT formula ϕ is satisfiable
  - Color all true literals T
  - Color node below green node F, and node below that B
  - Color remaining middle row nodes B
  - Color remaining bottom nodes T or F as forced